Name : Prajwal Mhase

ROll NO : 93

Div : B

In [34]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In [35]:
```
df=pd.read_csv('creditcard.csv')
```

In [36]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
 #   Column  Non-Null Count   Dtype  
---  ------  --------------   -----  
 0   Time    284807 non-null  float64
 1   V1      284807 non-null  float64
 2   V2      284807 non-null  float64
 3   V3      284807 non-null  float64
 4   V4      284807 non-null  float64
 5   V5      284807 non-null  float64
 6   V6      284807 non-null  float64
 7   V7      284807 non-null  float64
 8   V8      284807 non-null  float64
 9   V9      284807 non-null  float64
 10  V10     284807 non-null  float64
 11  V11     284807 non-null  float64
 12  V12     284807 non-null  float64
 13  V13     284807 non-null  float64
 14  V14     284807 non-null  float64
 15  V15     284807 non-null  float64
 16  V16     284807 non-null  float64
 17  V17     284807 non-null  float64
 18  V18     284807 non-null  float64
 19  V19     284807 non-null  float64
 20  V20     284807 non-null  float64
 21  V21     284807 non-null  float64
 22  V22     284807 non-null  float64
 23  V23     284807 non-null  float64
 24  V24     284807 non-null  float64
 25  V25     284807 non-null  float64
 26  V26     284807 non-null  float64
 27  V27     284807 non-null  float64
 28  V28     284807 non-null  float64
 29  Amount  284807 non-null  float64
 30  Class   284807 non-null  int64  
dtypes: float64(30), int64(1)
memory usage: 67.4 MB
```

In [37]:
```
df.head()
```

Out[37]:

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V22	
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	...	-0.018307	0.277838	-0.
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	...	-0.225775	-0.638672	0.
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	...	0.247998	0.771679	0.
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	...	-0.108300	0.005274	-0.
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	...	-0.009431	0.798278	-0.

5 rows × 31 columns

In [38]:
```
df.describe()
```

Out[38]:

	Time	V1	V2	V3	V4	V5	V6	V7	
count	284807.000000	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.8
mean	94813.859575	1.175161e-15	3.384974e-16	-1.379537e-15	2.094852e-15	1.021879e-15	1.494498e-15	-5.620335e-16	1.1
std	47488.145955	1.958696e+00	1.651309e+00	1.516255e+00	1.415869e+00	1.380247e+00	1.332271e+00	1.237094e+00	1.1
min	0.000000	-5.640751e+01	-7.271573e+01	-4.832559e+01	-5.683171e+00	-1.137433e+02	-2.616051e+01	-4.355724e+01	-7.3
25%	54201.500000	-9.203734e-01	-5.985499e-01	-8.903648e-01	-8.486401e-01	-6.915971e-01	-7.682956e-01	-5.540759e-01	-2.0
50%	84692.000000	1.810880e-02	6.548556e-02	1.798463e-01	-1.984653e-02	-5.433583e-02	-2.741871e-01	4.010308e-02	2.2
75%	139320.500000	1.315642e+00	8.037239e-01	1.027196e+00	7.433413e-01	6.119264e-01	3.985649e-01	5.704361e-01	3.2
max	172792.000000	2.454930e+00	2.205773e+01	9.382558e+00	1.687534e+01	3.480167e+01	7.330163e+01	1.205895e+02	2.0

8 rows × 31 columns

In [39]:
```
df.isnull().sum()
```

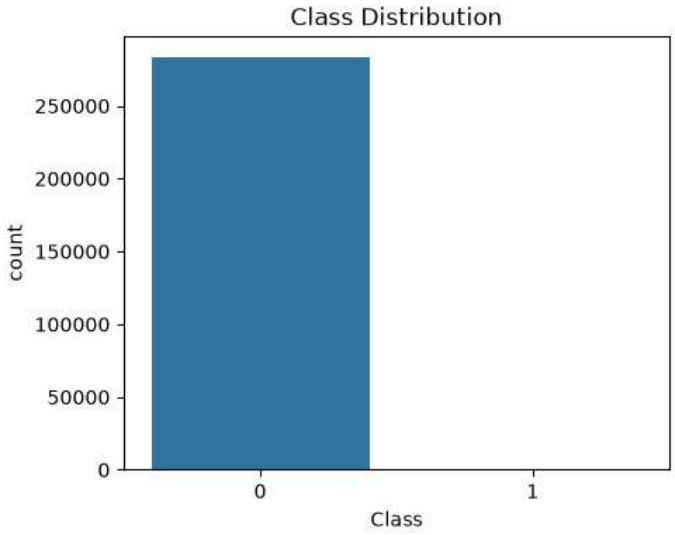
Out[39]:
```
Time      0
V1        0
V2        0
V3        0
V4        0
V5        0
V6        0
V7        0
V8        0
V9        0
V10       0
V11       0
V12       0
V13       0
V14       0
V15       0
V16       0
V17       0
V18       0
V19       0
V20       0
V21       0
V22       0
V23       0
V24       0
V25       0
V26       0
V27       0
V28       0
Amount    0
Class     0
dtype: int64
```

In [40]:
```
df = df.dropna().drop_duplicates()

df["Class"] = df["Class"].astype(int)
```

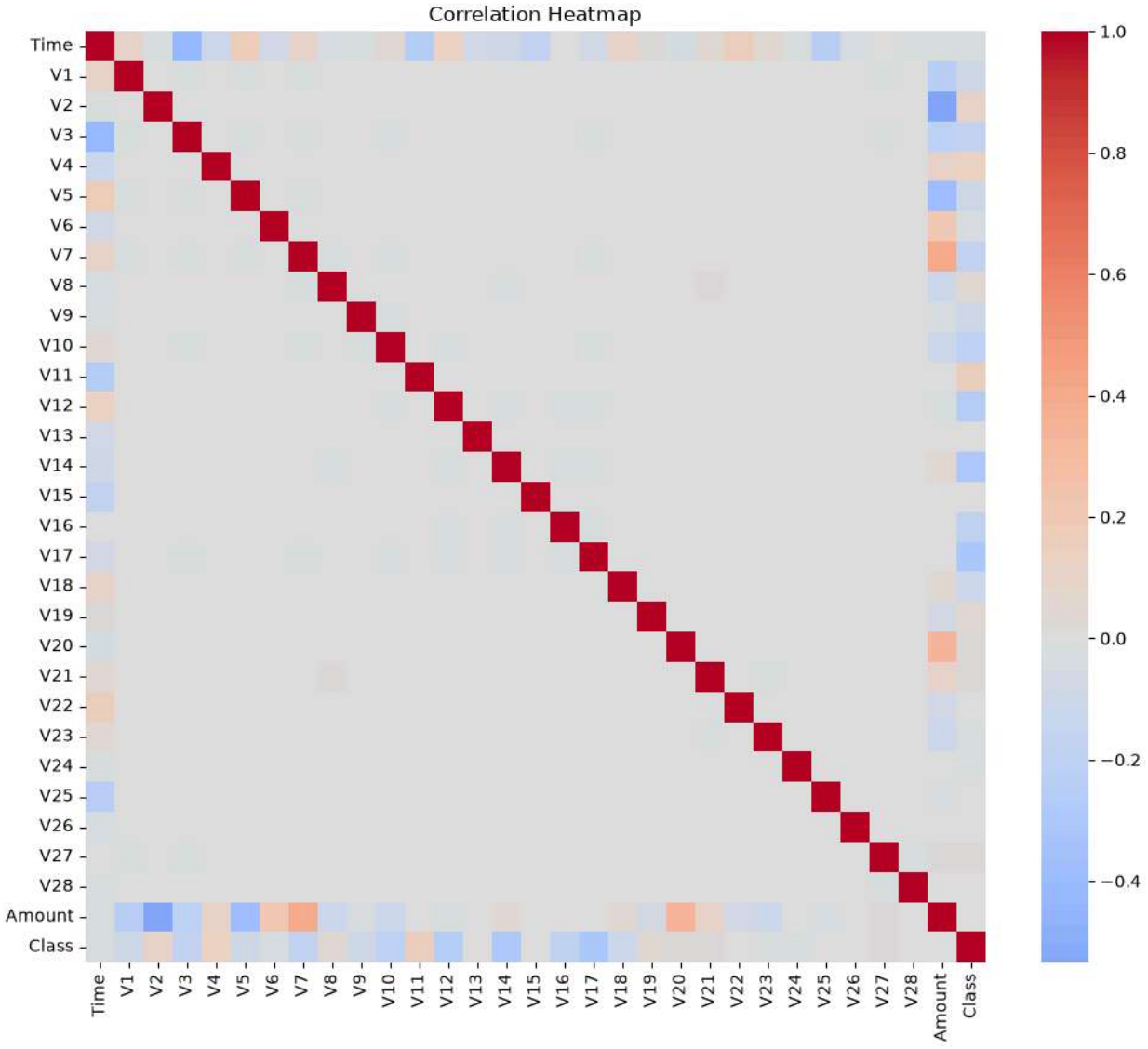
In [41]:
```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(5,4))
sns.countplot(data=df, x="Class")
plt.title("Class Distribution")
plt.show()
```



In [42]:
```
plt.figure(figsize=(12,10))
sns.heatmap(df.corr(), cmap="coolwarm", center=0)

plt.title("Correlation Heatmap")
plt.show()
```



In [43]:
```
X = df.drop("Class", axis=1)
y = df["Class"]

print("NaN in X :", X.isnull().sum().sum())
print("NaN in y :", y.isnull().sum())
```

```
NaN in X : 0
NaN in y : 0
```

In [44]:
```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

In [45]:
```python
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

In [46]:
```python
from imblearn.over_sampling import SMOTE

smote = SMOTE(random_state=42)

X_train, y_train = smote.fit_resample(
    X_train,
    y_train
)

print("After SMOTE :", X_train.shape, y_train.shape)
```

```
After SMOTE : (453204, 30) (453204,)
```

In [47]:
```python
from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout
from tensorflow.keras.layers import BatchNormalization

from tensorflow.keras.optimizers import Adam
from tensorflow.keras.optimizers import SGD
from tensorflow.keras.optimizers import RMSprop

from tensorflow.keras.callbacks import EarlyStopping

from tensorflow.keras.regularizers import l2
```

In [48]:
```python
def create_model(optimizer):

    model = Sequential()

    model.add(Dense(64,
                    activation="relu",
                    kernel_regularizer=l2(0.001),
                    input_shape=(X_train.shape[1],)))

    model.add(BatchNormalization())

    model.add(Dropout(0.30))

    model.add(Dense(32,
                    activation="relu",
                    kernel_regularizer=l2(0.001)))

    model.add(BatchNormalization())

    model.add(Dropout(0.30))

    model.add(Dense(16,
                    activation="relu"))

    model.add(Dropout(0.20))

    model.add(Dense(1,
                    activation="sigmoid"))

    model.compile(
        optimizer=optimizer,
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )

    return model
```

In [49]:
```
early = EarlyStopping(
    monitor="val_loss",
    patience=5,
    restore_best_weights=True
)
```

In [50]:
```
optimizers = {

    "Adam": Adam(0.001),

    "SGD": SGD(0.01),

    "RMSprop": RMSprop(0.001)

}
```

In [51]:
```
summary = []
```

In [52]:
```
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
from sklearn.metrics import roc_curve
from sklearn.metrics import auc
from sklearn.metrics import precision_recall_curve

for name, opt in optimizers.items():

    print("="*70)
    print("Training using", name)

    model = create_model(opt)

    history = model.fit(
        X_train,
        y_train,
        epochs=25,
        batch_size=256,
        validation_split=0.2,
        callbacks=[early]
    )

    # Accuracy Graph

    plt.figure(figsize=(6,4))

    plt.plot(history.history["accuracy"], label="Train")

    plt.plot(history.history["val_accuracy"], label="Validation")

    plt.title(f"{name} Accuracy")

    plt.legend()

    plt.grid(True)

    plt.show()

    # Loss Graph

    plt.figure(figsize=(6,4))

    plt.plot(history.history["loss"], label="Train")

    plt.plot(history.history["val_loss"], label="Validation")

    plt.title(f"{name} Loss")

    plt.legend()

    plt.grid(True)

    plt.show()

    # Prediction

    y_prob = model.predict(X_test)

    y_pred = (y_prob > 0.5).astype(int)

    # Metrics
```

```
    acc = accuracy_score(y_test, y_pred)

    pre = precision_score(y_test, y_pred)

    rec = recall_score(y_test, y_pred)

    f1 = f1_score(y_test, y_pred)

    print(classification_report(y_test, y_pred))

    # Confusion Matrix

    cm = confusion_matrix(y_test, y_pred)

    plt.figure(figsize=(5,4))

    sns.heatmap(cm,
                annot=True,
                fmt="d",
                cmap="Blues")

    plt.title(f"{name} Confusion Matrix")

    plt.xlabel("Predicted")

    plt.ylabel("Actual")

    plt.show()

    # ROC Curve

    fpr, tpr, _ = roc_curve(y_test, y_prob)

    roc_auc = auc(fpr, tpr)

    plt.figure(figsize=(6,4))

    plt.plot(fpr,
             tpr,
             label=f"AUC={roc_auc:.4f}")

    plt.plot([0,1],[0,1],"--")

    plt.title(f"{name} ROC Curve")

    plt.legend()

    plt.grid(True)

    plt.show()

    # Precision Recall Curve

    precision, recall, _ = precision_recall_curve(
        y_test,
        y_prob
    )

    plt.figure(figsize=(6,4))

    plt.plot(recall,
             precision)

    plt.title(f"{name} Precision Recall Curve")

    plt.xlabel("Recall")

    plt.ylabel("Precision")

    plt.grid(True)

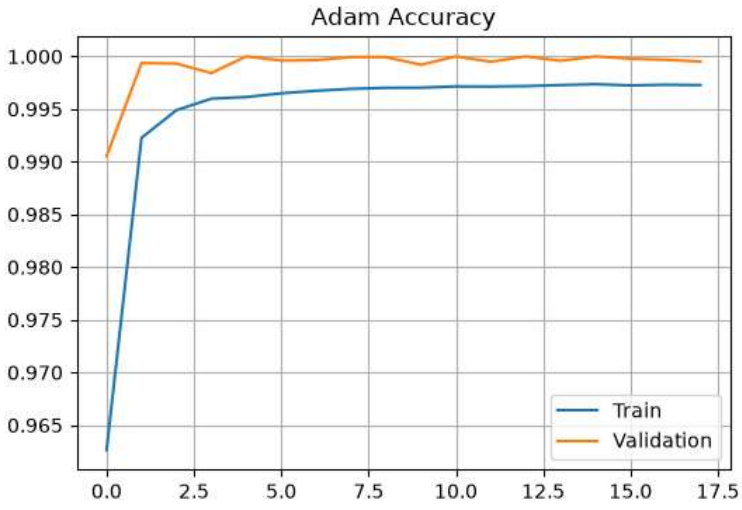
    plt.show()

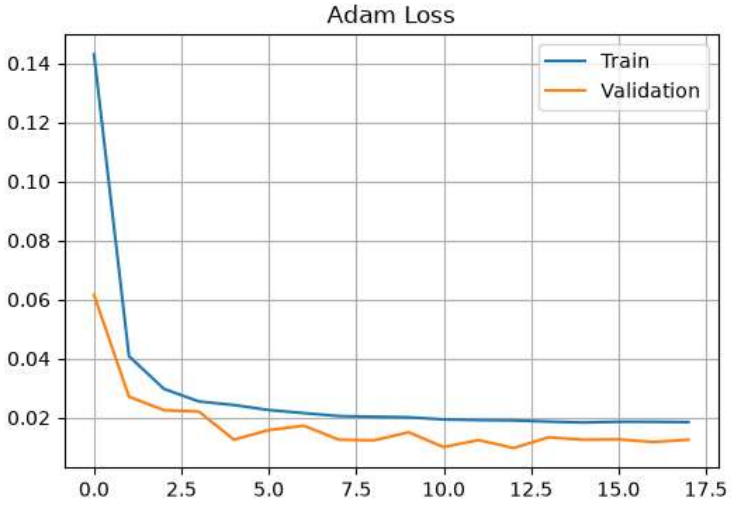
    summary.append([name, acc, pre, rec, f1])
```

```
======================================================================
Training using Adam
```

```
d:\Advanced data science Lab\.venv\Lib\site-packages\keras\src\layers\core\dense.py:107: UserWarning: Do not pass an `input
_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first l
ayer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
Epoch 1/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9626 - loss: 0.1430 - val_accuracy: 0.9905 - val_loss: 0.0616
Epoch 2/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9923 - loss: 0.0408 - val_accuracy: 0.9994 - val_loss: 0.0271
Epoch 3/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9949 - loss: 0.0297 - val_accuracy: 0.9993 - val_loss: 0.0225
Epoch 4/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9960 - loss: 0.0255 - val_accuracy: 0.9984 - val_loss: 0.0220
Epoch 5/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9961 - loss: 0.0243 - val_accuracy: 1.0000 - val_loss: 0.0125
Epoch 6/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9965 - loss: 0.0226 - val_accuracy: 0.9996 - val_loss: 0.0158
Epoch 7/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9967 - loss: 0.0215 - val_accuracy: 0.9996 - val_loss: 0.0173
Epoch 8/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9969 - loss: 0.0205 - val_accuracy: 0.9999 - val_loss: 0.0126
Epoch 9/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9970 - loss: 0.0203 - val_accuracy: 0.9999 - val_loss: 0.0123
Epoch 10/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9970 - loss: 0.0201 - val_accuracy: 0.9992 - val_loss: 0.0150
Epoch 11/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9972 - loss: 0.0194 - val_accuracy: 1.0000 - val_loss: 0.0100
Epoch 12/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9971 - loss: 0.0192 - val_accuracy: 0.9995 - val_loss: 0.0124
Epoch 13/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9972 - loss: 0.0191 - val_accuracy: 1.0000 - val_loss: 0.0097
Epoch 14/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9973 - loss: 0.0186 - val_accuracy: 0.9996 - val_loss: 0.0133
Epoch 15/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9974 - loss: 0.0184 - val_accuracy: 1.0000 - val_loss: 0.0125
Epoch 16/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9972 - loss: 0.0186 - val_accuracy: 0.9998 - val_loss: 0.0126
Epoch 17/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9973 - loss: 0.0186 - val_accuracy: 0.9997 - val_loss: 0.0118
Epoch 18/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9973 - loss: 0.0185 - val_accuracy: 0.9995 - val_loss: 0.0125
```

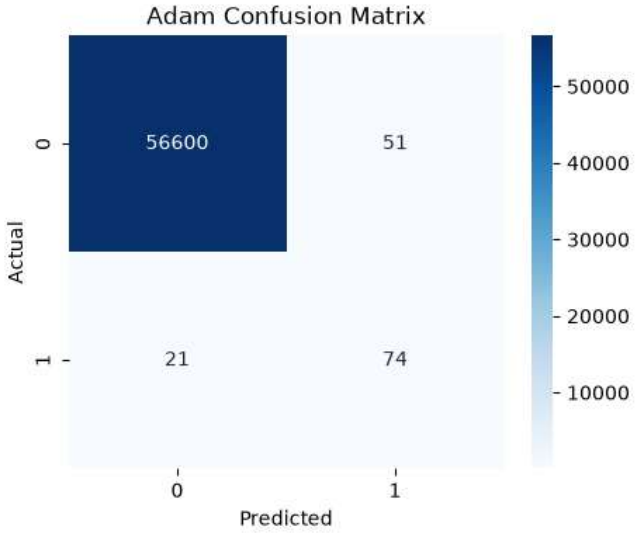


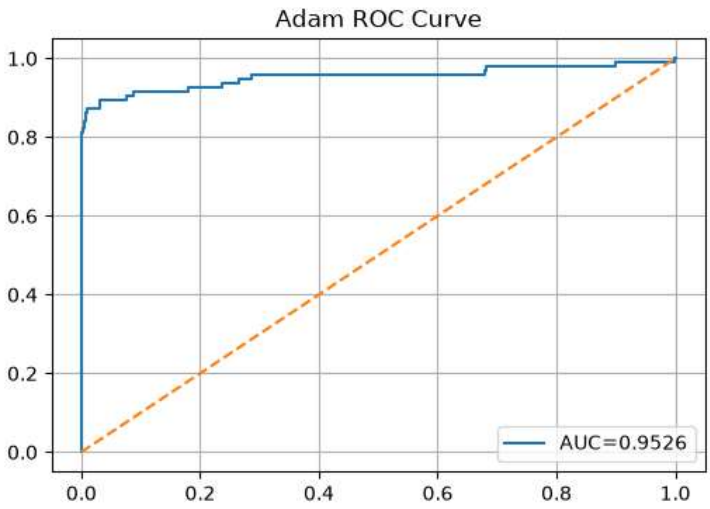


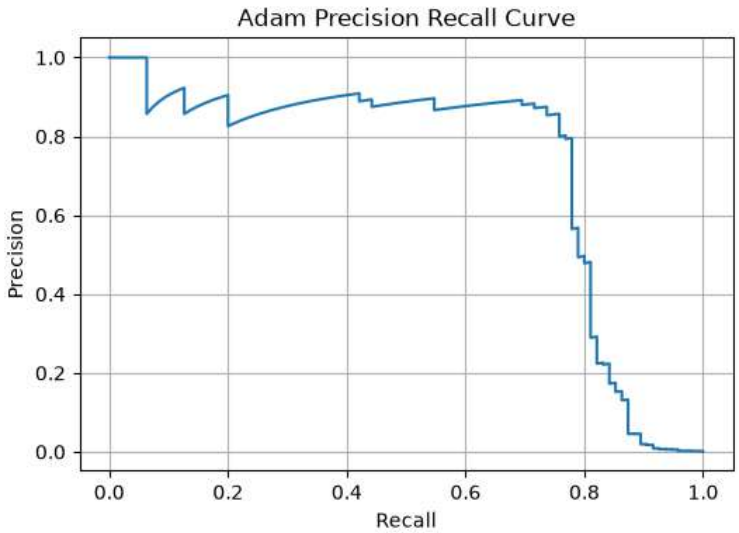
```
1774/1774 ━━━━━━━━━━━━━━━━━━━━ 2s 826us/step
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     56651
           1       0.59      0.78      0.67        95

    accuracy                           1.00     56746
   macro avg       0.80      0.89      0.84     56746
weighted avg       1.00      1.00      1.00     56746
```



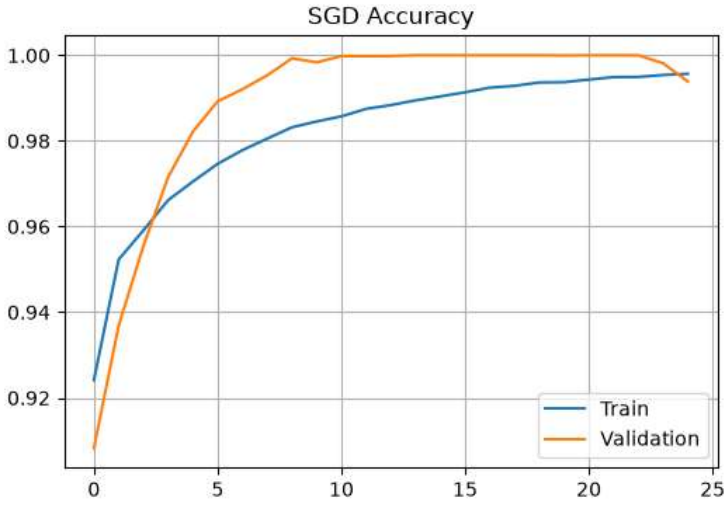


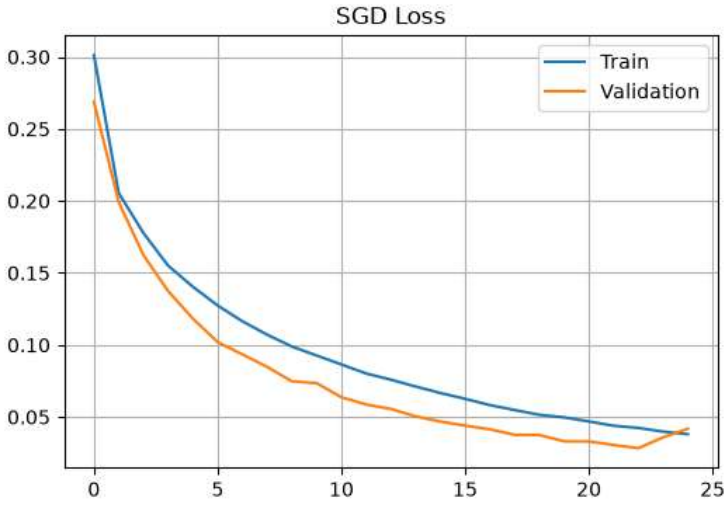



```
======================================================================
Training using SGD
Epoch 1/25
```

```
d:\Advanced data science Lab\.venv\Lib\site-packages\keras\src\layers\core\dense.py:107: UserWarning: Do not pass an `input
_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first l
ayer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9242 - loss: 0.3011 - val_accuracy: 0.9082 - val_loss: 0.2688
Epoch 2/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9523 - loss: 0.2055 - val_accuracy: 0.9369 - val_loss: 0.1992
Epoch 3/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9591 - loss: 0.1777 - val_accuracy: 0.9555 - val_loss: 0.1623
Epoch 4/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9662 - loss: 0.1550 - val_accuracy: 0.9716 - val_loss: 0.1374
Epoch 5/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9705 - loss: 0.1404 - val_accuracy: 0.9822 - val_loss: 0.1182
Epoch 6/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9746 - loss: 0.1274 - val_accuracy: 0.9893 - val_loss: 0.1020
Epoch 7/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9778 - loss: 0.1164 - val_accuracy: 0.9921 - val_loss: 0.0934
Epoch 8/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9805 - loss: 0.1071 - val_accuracy: 0.9953 - val_loss: 0.0848
Epoch 9/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9832 - loss: 0.0989 - val_accuracy: 0.9993 - val_loss: 0.0747
Epoch 10/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9845 - loss: 0.0927 - val_accuracy: 0.9984 - val_loss: 0.0735
Epoch 11/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9857 - loss: 0.0864 - val_accuracy: 0.9998 - val_loss: 0.0637
Epoch 12/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9875 - loss: 0.0802 - val_accuracy: 0.9998 - val_loss: 0.0586
Epoch 13/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9884 - loss: 0.0759 - val_accuracy: 0.9998 - val_loss: 0.0555
Epoch 14/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9895 - loss: 0.0711 - val_accuracy: 1.0000 - val_loss: 0.0504
Epoch 15/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9904 - loss: 0.0666 - val_accuracy: 1.0000 - val_loss: 0.0468
Epoch 16/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9914 - loss: 0.0626 - val_accuracy: 1.0000 - val_loss: 0.0440
Epoch 17/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9924 - loss: 0.0582 - val_accuracy: 1.0000 - val_loss: 0.0414
Epoch 18/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9929 - loss: 0.0548 - val_accuracy: 1.0000 - val_loss: 0.0376
Epoch 19/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9937 - loss: 0.0515 - val_accuracy: 1.0000 - val_loss: 0.0375
Epoch 20/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9937 - loss: 0.0497 - val_accuracy: 1.0000 - val_loss: 0.0331
Epoch 21/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9943 - loss: 0.0469 - val_accuracy: 1.0000 - val_loss: 0.0330
Epoch 22/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9949 - loss: 0.0439 - val_accuracy: 1.0000 - val_loss: 0.0305
Epoch 23/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9949 - loss: 0.0424 - val_accuracy: 1.0000 - val_loss: 0.0283
Epoch 24/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9954 - loss: 0.0398 - val_accuracy: 0.9981 - val_loss: 0.0358
Epoch 25/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9957 - loss: 0.0381 - val_accuracy: 0.9939 - val_loss: 0.0419
```

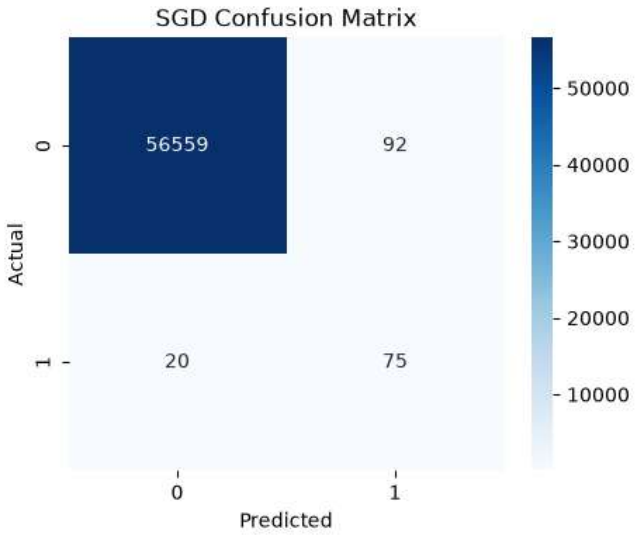


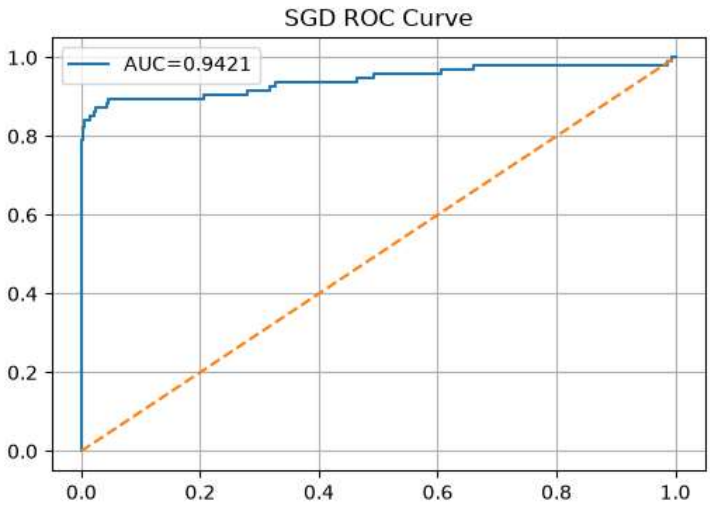


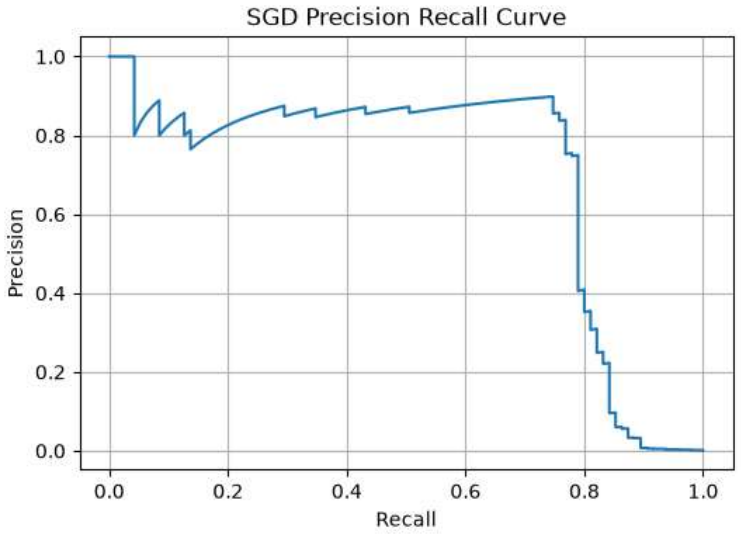

```
1774/1774 ━━━━━━━━━━━━━━━━━━━━ 2s 828us/step
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     56651
           1       0.45      0.79      0.57        95

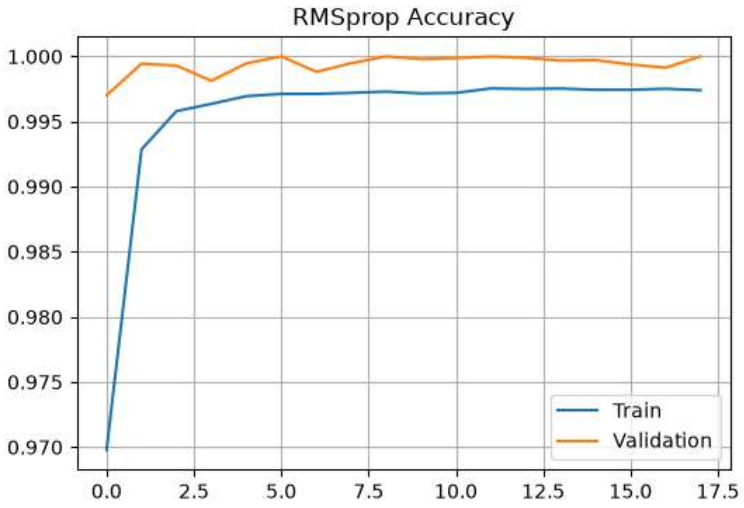
    accuracy                           1.00     56746
   macro avg       0.72      0.89      0.79     56746
weighted avg       1.00      1.00      1.00     56746
```

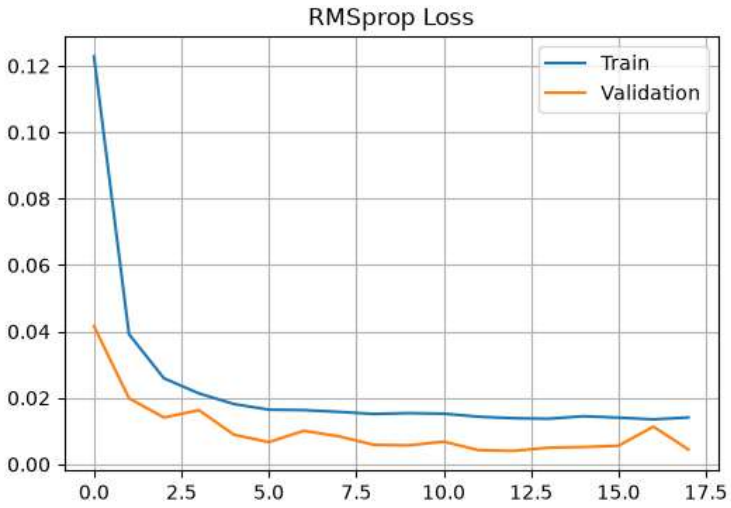







```
======================================================================
Training using RMSprop
Epoch 1/25
d:\Advanced data science Lab\.venv\Lib\site-packages\keras\src\layers\core\dense.py:107: UserWarning: Do not pass an `input
_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first l
ayer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.9698 - loss: 0.1228 - val_accuracy: 0.9970 - val_loss: 0.0416
Epoch 2/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9929 - loss: 0.0392 - val_accuracy: 0.9994 - val_loss: 0.0199
Epoch 3/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9958 - loss: 0.0259 - val_accuracy: 0.9993 - val_loss: 0.0141
Epoch 4/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9964 - loss: 0.0213 - val_accuracy: 0.9981 - val_loss: 0.0163
Epoch 5/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9969 - loss: 0.0181 - val_accuracy: 0.9995 - val_loss: 0.0089
Epoch 6/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9971 - loss: 0.0165 - val_accuracy: 1.0000 - val_loss: 0.0067
Epoch 7/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 5s 2ms/step - accuracy: 0.9971 - loss: 0.0163 - val_accuracy: 0.9988 - val_loss: 0.0101
Epoch 8/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9972 - loss: 0.0158 - val_accuracy: 0.9995 - val_loss: 0.0084
Epoch 9/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9973 - loss: 0.0152 - val_accuracy: 1.0000 - val_loss: 0.0059
Epoch 10/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9972 - loss: 0.0154 - val_accuracy: 0.9998 - val_loss: 0.0057
Epoch 11/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.9972 - loss: 0.0152 - val_accuracy: 0.9999 - val_loss: 0.0068
Epoch 12/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.9975 - loss: 0.0143 - val_accuracy: 1.0000 - val_loss: 0.0043
Epoch 13/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9975 - loss: 0.0139 - val_accuracy: 0.9999 - val_loss: 0.0040
Epoch 14/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9975 - loss: 0.0137 - val_accuracy: 0.9997 - val_loss: 0.0050
Epoch 15/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9974 - loss: 0.0145 - val_accuracy: 0.9997 - val_loss: 0.0052
Epoch 16/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9974 - loss: 0.0140 - val_accuracy: 0.9994 - val_loss: 0.0056
Epoch 17/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9975 - loss: 0.0135 - val_accuracy: 0.9991 - val_loss: 0.0113
Epoch 18/25
1417/1417 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9974 - loss: 0.0141 - val_accuracy: 1.0000 - val_loss: 0.0044
```

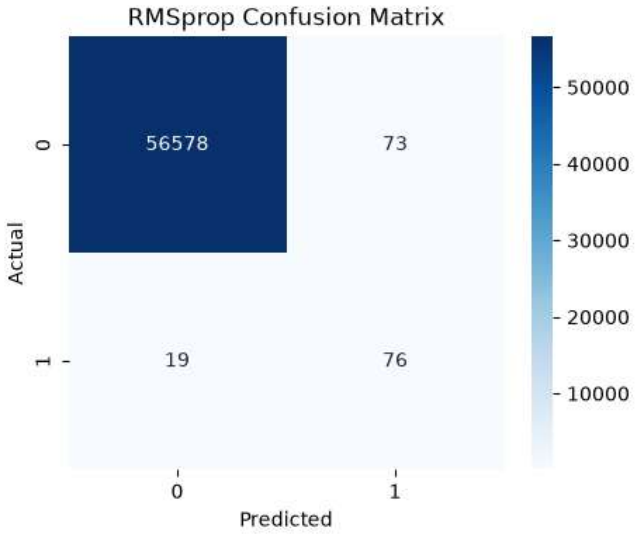


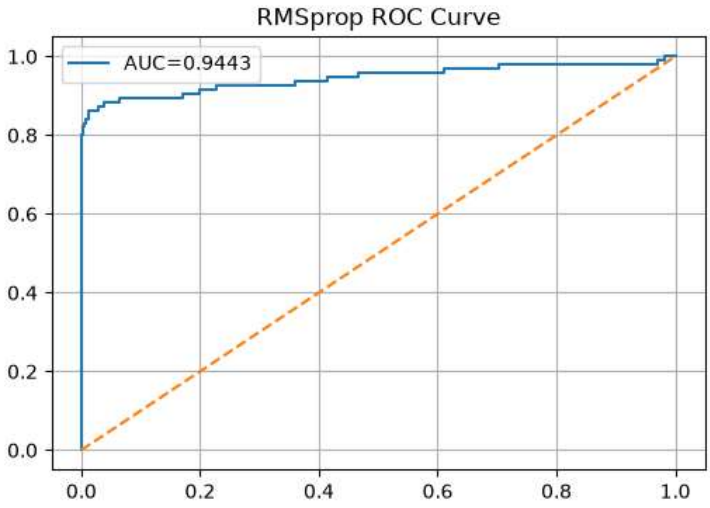


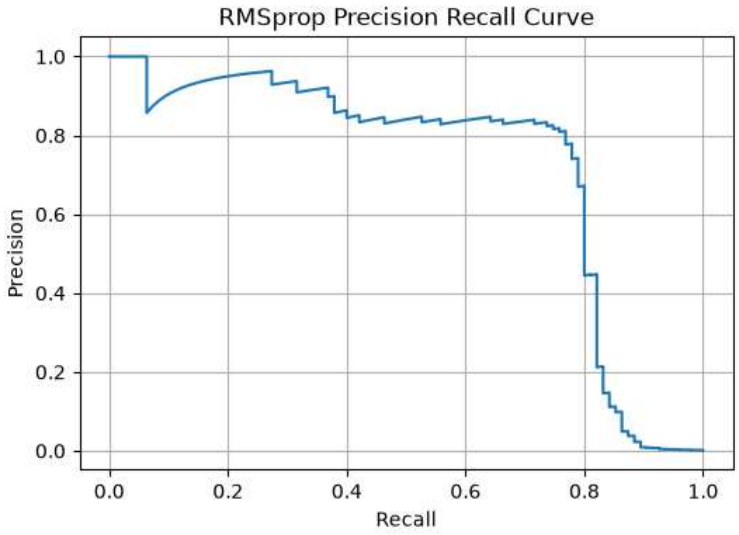

```
1774/1774 ━━━━━━━━━━━━━━━━━━━━ 2s 839us/step
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     56651
           1       0.51      0.80      0.62        95

    accuracy                           1.00     56746
   macro avg       0.75      0.90      0.81     56746
weighted avg       1.00      1.00      1.00     56746
```







In [53]:
```
import pandas as pd

result = pd.DataFrame(
    summary,
    columns=[
        "Optimizer",
        "Accuracy",
        "Precision",
        "Recall",
        "F1 Score"
    ]
)

print(result)
```

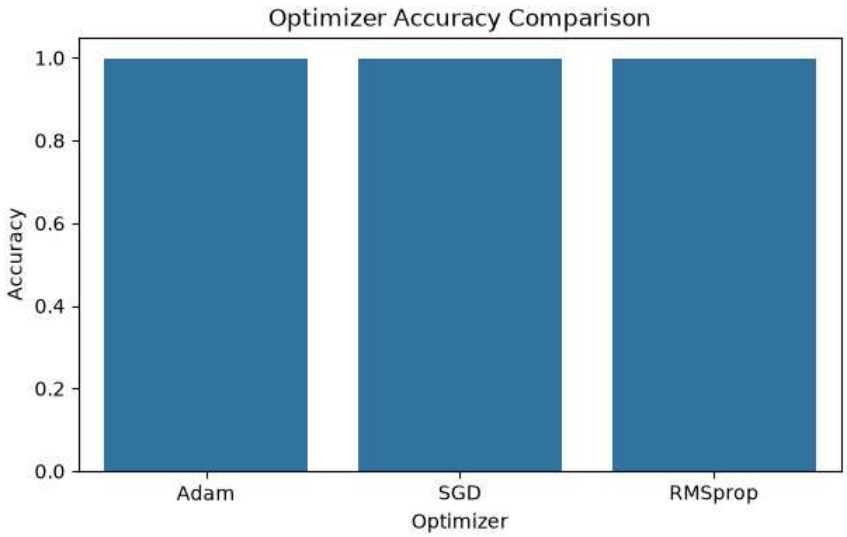
```
  Optimizer  Accuracy  Precision    Recall  F1 Score
0      Adam  0.998731   0.592000  0.778947  0.672727
1       SGD  0.998026   0.449102  0.789474  0.572519
2   RMSprop  0.998379   0.510067  0.800000  0.622951
```

In [54]:
```
plt.figure(figsize=(7,4))

sns.barplot(
    data=result,
    x="Optimizer",
    y="Accuracy"
)

plt.title("Optimizer Accuracy Comparison")

plt.show()
```


Optimizer Accuracy Comparison
1.0
0.8
0.6
0.4
0.2
0.0
Accuracy
Adam
SGD
RMSprop
Optimizer

In [ ]: